

SPRING REST HANDSON EXERCISES

Hands on 1: Create a Spring Web Project using Maven

Follow the steps to configure and build the application:

1. Generate a project from spring initializer with package **com.sasank.spring_learn**.
2. Include dependencies: Spring Web, Spring Boot DevTools.
3. Run the application main class locally. Verify Tomcat initializes on port 8085.

SpringLearnApplication Console Output:

```
Command Prompt

.\ /___.-._.-_-( )_-__/_\ \ \ \
( ( )\_ |'-'|'|_|'-V_-| \ \ \ \
\ \ _ | |_) | | | | | | ( | | ) ) )
' | _ | ._| | | | | | _ \ , | / / / /
=====|_|=====|___/_/_/_/_/

:: Spring Boot ::                (v3.1.2)


2026-07-24T19:40:02.124+05:30 INFO 14208 --- [spring-learn] [main] c.s.s.SpringLearnApplication : St
2026-07-24T19:40:02.129+05:30 INFO 14208 --- [spring-learn] [main] c.s.s.SpringLearnApplication : No
2026-07-24T19:40:03.541+05:30 INFO 14208 --- [spring-learn] [main] o.s.b.w.embedded.tomcat.TomcatWebServer : T
2026-07-24T19:40:03.558+05:30 INFO 14208 --- [spring-learn] [main] o.apache.catalina.core.StandardService : S
2026-07-24T19:40:03.559+05:30 INFO 14208 --- [spring-learn] [main] o.apache.catalina.core.StandardEngine : S
2026-07-24T19:40:03.681+05:30 INFO 14208 --- [spring-learn] [main] o.a.c.c.C.[Tomcat].[localhost].[/] : I
2026-07-24T19:40:03.682+05:30 INFO 14208 --- [spring-learn] [main] w.s.c.ServletWebServerApplicationContext : R
2026-07-24T19:40:04.225+05:30 INFO 14208 --- [spring-learn] [main] o.s.b.w.embedded.tomcat.TomcatWebServer : T
2026-07-24T19:40:04.238+05:30 INFO 14208 --- [spring-learn] [main] c.s.s.SpringLearnApplication : St
```

Hands on 2: Configure Application Context

We verify context injection and SimpleDateFormat class loaders using spring bean config XML files.

File: **date-format.xml**

```
<?xml version="1.0" encoding="UTF-8"?> <beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
https://www.springframework.org/schema/beans/spring-beans.xsd"> <bean id="dateFormat"
class="java.text.SimpleDateFormat"> <constructor-arg value="dd/MM/yyyy"/> </bean> </beans>
```

File: **SpringLearnApplication.java**

```
package com.sasank.spring_learn; import java.text.SimpleDateFormat; import java.util.Date; import
org.springframework.boot.SpringApplication; import
org.springframework.boot.autoconfigure.SpringBootApplication; import
org.springframework.context.ApplicationContext; import
org.springframework.context.support.ClassPathXmlApplicationContext; @SpringBootApplication public
class SpringLearnApplication { public static void main(String[] args) {
SpringApplication.run(SpringLearnApplication.class, args); displayDate(); } public static void
displayDate() { ApplicationContext context = new
ClassPathXmlApplicationContext("date-format.xml"); SimpleDateFormat format =
context.getBean("dateFormat", SimpleDateFormat.class); System.out.println("Current Date: " +
format.format(new Date())); } }
```

Hands on 3: Port Configuration & Troubleshooting

Configure custom application port parameters inside **application.properties**:

```
server.port=8085
logging.level.com.sasank.spring_learn=debug
```

If the Tomcat server port is occupied, the application context fails to refresh with a `PortInUseException`, as shown below:

```
Command Prompt

2026-07-24T19:42:15.110+05:30 INFO 14882 --- [spring-learn] [main] o.s.b.w.embedded.tomcat.TomcatWebServer : T
2026-07-24T19:42:15.120+05:30 INFO 14882 --- [spring-learn] [main] o.apache.catalina.core.StandardService : S
2026-07-24T19:42:15.121+05:30 INFO 14882 --- [spring-learn] [main] o.apache.catalina.core.StandardEngine : S
2026-07-24T19:42:15.223+05:30 INFO 14882 --- [spring-learn] [main] o.a.c.c.C.[Tomcat].[localhost].[/] : I
2026-07-24T19:42:15.340+05:30 WARN 14882 --- [spring-learn] [main] w.s.c.ServletWebServerApplicationContext : E
2026-07-24T19:42:15.361+05:30 INFO 14882 --- [spring-learn] [main] o.apache.catalina.core.StandardService : S
2026-07-24T19:42:15.385+05:30 ERROR 14882 --- [spring-learn] [main] o.s.b.d.LoggingFailureAnalysisReporter :
*****
APPLICATION FAILED TO START
*****

Description:
Web server failed to start. Port 8085 was already in use.

Action:
Identify and stop the process that's listening on port 8085 or configure this application to listen on another port.
```

Hands on 4: Controller Mappings & REST Endpoints

Define standard REST Controllers to support greetings and country configurations.

File: **HelloController.java**

```
package com.sasank.spring_learn.controller; import org.slf4j.Logger; import
org.slf4j.LoggerFactory; import org.springframework.web.bind.annotation.GetMapping; import
org.springframework.web.bind.annotation.RestController; @RestController public class
HelloController { private static final Logger LOGGER =
LoggerFactory.getLogger(HelloController.class); @GetMapping("/hello") public String sayHello() {
LOGGER.info("START - sayHello"); String message = "Hello from Sasank's Spring Boot App!";
LOGGER.info("END - sayHello"); return message; } }
```

Execution Results (/hello):

A screenshot of a web browser window. The address bar shows the URL "http://localhost:8085/hello". The browser window has a light gray header bar with two small circular icons on the left.

Hello from Sasank's Spring Boot App!

File: **Country.java**

```
package com.sasank.spring_learn; public class Country { private String code; private String name;
public Country() {} public String getCode() { return code; } public void setCode(String code) {
this.code = code; } public String getName() { return name; } public void setName(String name) {
this.name = name; } }
```

File: **country.xml**

```
<bean id="country" class="com.sasank.spring_learn.Country"> <property name="code" value="IN"/>
<property name="name" value="India"/> </bean> <bean id="us"
class="com.sasank.spring_learn.Country"> <property name="code" value="US"/> <property name="name"
value="United States"/> </bean> <bean id="ca" class="com.sasank.spring_learn.Country"> <property
name="code" value="CA"/> <property name="name" value="Canada"/> </bean> <bean id="fr"
class="com.sasank.spring_learn.Country"> <property name="code" value="FR"/> <property name="name"
value="France"/> </bean>
```

Hands on 5: Country Controller Mappings

File: **CountryController.java**

```
package com.sasank.spring_learn.controller; import
org.springframework.context.ApplicationContext; import
org.springframework.context.support.ClassPathXmlApplicationContext; import
org.springframework.web.bind.annotation.GetMapping; import
org.springframework.web.bind.annotation.RestController; import com.sasank.spring_learn.Country;
import java.util.List; @RestController public class CountryController { @GetMapping("/country")
public Country getCountryIndia() { ApplicationContext context = new
ClassPathXmlApplicationContext("country.xml"); return context.getBean("country", Country.class);
} @GetMapping("/countries") public List getAllCountries() { ApplicationContext context = new
ClassPathXmlApplicationContext("country.xml"); return context.getBean("countryList", List.class);
} }
```

Execution Results (/country):

http://localhost:8085/country

```
{ "code": "IN", "name": "India" }
```

Execution Results (/countries):

http://localhost:8085/countries

```
[{"code": "IN", "name": "India"}, {"code": "US", "name": "United States"}, {"code": "CA", "name": "Canada"}, {"code": "FR", "name": "France"}]
```

Hands on 6: Parameterized Route Retrieval and Service Layer

File: **CountryService.java**

```
package com.sasank.spring_learn.service; import java.util.List; import
org.springframework.context.ApplicationContext; import
org.springframework.context.support.ClassPathXmlApplicationContext; import
org.springframework.stereotype.Service; import com.sasank.spring_learn.Country; @Service public
class CountryService { public Country getCountry(String code) { ApplicationContext context = new
ClassPathXmlApplicationContext("country.xml"); List list = context.getBean("countryList",
List.class); return list.stream() .filter(c -> c.getCode().equalsIgnoreCase(code)) .findFirst()
.orElseThrow(() -> new RuntimeException("Country Not Found")); } }
```

Execution Results (/country/us):

A screenshot of a web browser window. The address bar shows the URL "http://localhost:8085/country/us". There are two small circular icons to the left of the address bar.

```
{"code":"US","name":"United States"}
```

JWT Authentication Hands-On

Hands on 7: Basic Security Filter Chains & JWT Tokens

Verify basic authentication and unauthorized requests using curl CLI commands executed on the system path:

1. Unauthenticated Request (/countries):

Command Prompt

```
C:\Users\sasan> curl -v http://localhost:8085/countries
* Trying 127.0.0.1:8085...
* Connected to localhost (127.0.0.1) port 8085
> GET /countries HTTP/1.1
> Host: localhost:8085
> User-Agent: curl/8.0.1
> Accept: */*

< HTTP/1.1 401 Unauthorized
< WWW-Authenticate: Basic realm="Realm"
< Content-Length: 0
< Date: Fri, 24 Jul 2026 14:15:32 GMT
< Connection: keep-alive
```

2. Authenticated Request with Credentials:

Command Prompt

```
C:\Users\sasan> curl -u user:pwd http://localhost:8085/countries
[{"code":"IN","name":"India"}, {"code":"US","name":"United States"}, {"code":"CA","name":"Canada"}, {"code":"FR","name":"France"}]
```

3. Retrieve JWT Token via Credentials:

Command Prompt

```
C:\Users\sasan> curl -u user:pwd http://localhost:8085/authenticate
{"token":"eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiI1c2VyIiwiaWF0IjoxNzE2NTYwMTI4LCJleHAiOiE3MTY1NjM3Mjh9.Vw8w8W1vL60lC91xS7r_S6Vf3-7
```

4. Retrieve Data using Bearer Token:

Command Prompt

```
C:\Users\sasan> curl -H "Authorization: Bearer eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiI1c2VyIiwiaWF0IjoxNzE2NTYwMTI4LCJleHAiOiE3MTY1NjM3Mjh9.Vw8w8W1vL60lC91xS7r_S6Vf3-7" http://localhost:8085/data
[{"code":"IN","name":"India"}, {"code":"US","name":"United States"}, {"code":"CA","name":"Canada"}, {"code":"FR","name":"France"}]
```

File: SecurityConfig.java

```
package com.sasank.spring_learn.security; import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration; import
org.springframework.security.config.Customizer; import
org.springframework.security.config.annotation.web.builders.HttpSecurity; import
org.springframework.security.core.userdetails.User; import
org.springframework.security.core.userdetails.UserDetails; import
org.springframework.security.provisioning.InMemoryUserDetailsManager; import
org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder; import
org.springframework.security.crypto.password.PasswordEncoder; import
org.springframework.security.web.SecurityFilterChain; @Configuration public class SecurityConfig {
@Bean public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
http.httpBasic(Customizer.withDefaults()) .csrf(csrf -> csrf.disable())
.authorizeHttpRequests(auth -> auth .requestMatchers("/authenticate").permitAll()
.anyRequest().authenticated()); return http.build(); } }
```